
LECTURE NOTES

Lecture 15: Discrete Optimization

University

Simon Holm
April - 2026



DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

1	Discrete Optimization	3
2	Discrete vs Combinatorial Optimization	3
3	Integer Linear Programming (ILP)	4
3.1	Mathematical Programming: Modeling	4
3.2	Rounding	4
3.3	Cutting Planes	4
3.4	Chvatal-Gomory's Cutting Plane Algorithm	5
4	Branch and Bound	5
4.1	Bounding	6
4.2	Pruning	6

1 Discrete Optimization

- Integer (linear) programming
- Exact solution techniques
 - Dedicated algorithms
 - Dynamic programming
 - Constraint programming
 - SAT solvers
- Approximation solution techniques
 - Approximation algorithms
 - Greedy algorithms and local search
 - Metaheuristics, Randomized optimization algorithms
 - Mathheuristics
 - ... many more

2 Discrete vs Combinatorial Optimization

In Combinatorial Optimization, variables have some combinatorial structure, ie, sets, permutations, paths

Definition (Combinatorial Optimization Problem (COP))

Input: Given a finite set $N = \{1, \dots, n\}$ of objects, weights c_j for all $j \in N$, a collection $\mathcal{F}$ of feasible subsets of N **Task:** Find a minimum weight feasible subset, ie,

$$\min_{\{S \in \mathcal{F}\}} \sum_{j \in S} c_j$$

COP can also be modelled as discrete optimization problems. Typically: **incidence vector** of S , $x^S \in \mathbb{B}^n : x_j^S = \begin{cases} 1 & j \in S \\ 0 & \text{otherwise} \end{cases}$

Solving problems with variables that are discrete instead of continuous

Example

- | | |
|-------------------------------------|---------------------------------|
| • Set covering and set partitioning | • Bin packing problem |
| • Knapsack problem | • Graph coloring problem |
| • Traveling salesman problem | • Maximum flow problem |
| • Vehicle routing problem | • Minimum spanning tree problem |
| • Job scheduling problem | • Shortest path problem |
| • Facility location problem | • Steiner tree problem |
| | • Hamiltonian path problem |

3 Integer Linear Programming (ILP)

Linear Objective • Linear Constraints • but! integer variables

$$\begin{array}{llll}
 \max c^T x & \max c^T x & \max c^T x & \max c^T x + h^T y \\
 Ax \leq b & Ax \leq b & Ax \leq b & Ax + Gy \leq b \\
 x \geq 0 & x \geq 0 & x \in \{0,1\}^n & x \geq 0 \\
 & x \text{ integer} & & y \geq 0 \\
 & & & y \text{ integer}
 \end{array}$$

Linear Programming (LP) Integer Linear Programming (ILP) Binary Integer Program (BIP) Mixed Integer Linear Programming (MILP)
 0/1 Integer Programming

$$\begin{array}{ll}
 \max f(x) & \\
 g(x) \leq b & \text{Integer Non-linear} \\
 x \geq 0 & \text{Programming (MINLP)} \\
 x \text{ integer} &
 \end{array}$$

Figure 2: Different kinds of integer linear programming

3.1 Mathematical Programming: Modeling

Find out exactly what the decision maker needs to know: (common to use binary variables $\mathbb{B}$, but not always)

- which investments to make
- which routes to take
- which items to pack
- which job j should a person i be assigned to?

3.2 Rounding

One can relax a integer constraint to a linear one, solve the resulting linear program, and round the solution to an integer one. This is called **rounding**.

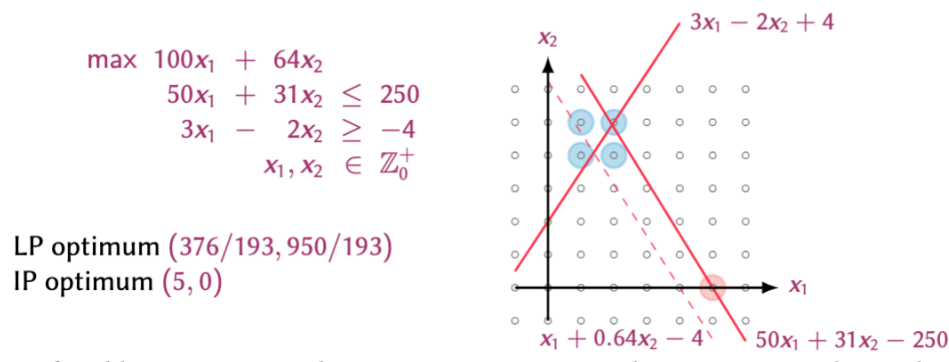


Figure 3: Rounding a linear program solution to an integer one can easily result in a very bad solution.

3.3 Cutting Planes

Lets tighten the linear programming relaxation by adding additional constraints, called **cutting planes**, that cut off the fractional solution but do not cut off any integer solutions.

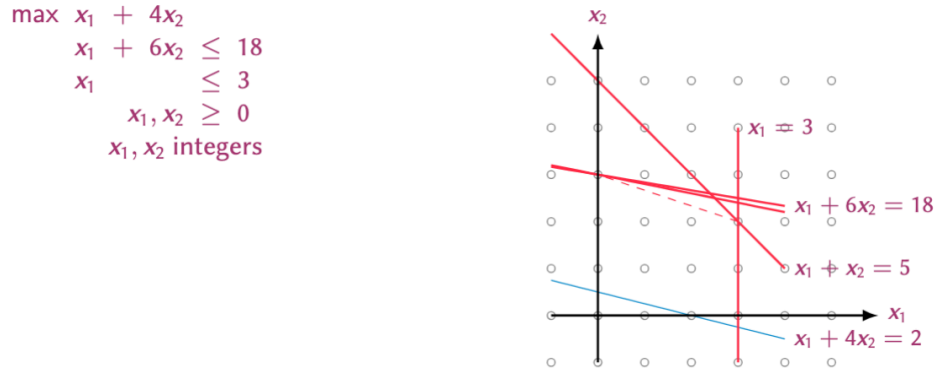


Figure 4: Cutting planes can be used to tighten the linear programming relaxation.

3.4 Chvatal-Gomory's Cutting Plane Algorithm

From the simplex we know that we can write the constraint $Ax = b$ as

$$\begin{aligned} A_B x_B^* + A_N x_N^* &= b \\ x_B^* + [A_B^{-1} A_N] x_N^* &= A_B^{-1} b \\ x_B^* + [A_B^{-1} A_N] x_N^* &= [A_B^{-1} b] \\ \text{since } x_B^* &= A_B^{-1} b - A_B^{-1} A_N x_N^* \\ A_B^{-1} b - [A_B^{-1} b] - (A_B^{-1} A_N - [A_B^{-1} A_N]) x_N^* &\leq 0 \end{aligned}$$

Using the method of Gomory's cut, we can add an additional inequality constraint for each nonintegral dimension

$$x_b^* - [x_b^*] - \sum_{j \in N} (|A|_{bj} - [A|_{bj}]) x_j^* \leq 0 \quad |A| = A_B^{-1} A_N$$

This will cut off the relaxed solution, but not any integer solutions.

$$\underbrace{x_b^* - [x_b^*]}_{>0} - \underbrace{\sum_{j \in N} (|A|_{bj} - [A|_{bj}]) x_j^*}_{=0} > 0$$

4 Branch and Bound

Branch and bound is a general algorithm for finding optimal solutions in a large set of solution possibilities.

It is a divide-and-conquer algorithm that systematically explores the solution space by branching it into smaller subproblems and bounding the optimal solution of each subproblem to eliminate suboptimal solutions.

Consider the problem $z = \min\{c^\top x : x \in S\}$

Then let $S = S_1 \cup S_2 \cup \dots \cup S_k$ be a composition of S into smaller subsets, and $z^k = \min\{c^\top x : x \in S_k\}$. Then $z = \min_k z^k$

For instance if $S \subseteq \{0, 1\}^3$, the the algorithm is as follows:

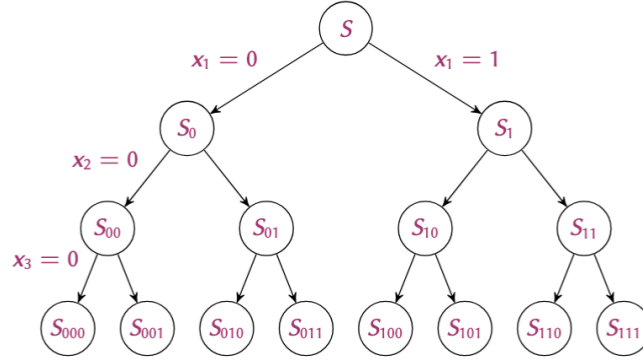


Figure 5: Branch and bound algorithm for a binary optimization problem $S \subseteq \{0, 1\}^3$.

4.1 Bounding

Consider a minimization problem with the following bounds

- Let $\underline{z}^k$ be a **lower bound** on z^k (dual bound)
- Let $|z|^k$ be a **upper bound** on z^k (primal bound)

So that $\underline{z}^k \leq z^k \leq |z|^k$

Then

- $\underline{z} = \min_k \underline{z}^k$ is a **lower bound** on z
- $|z| = \min_k |z|^k$ is a **upper bound** on z

4.2 Pruning

Only explore optimal paths by pruning (stop further explorations) suboptimal paths

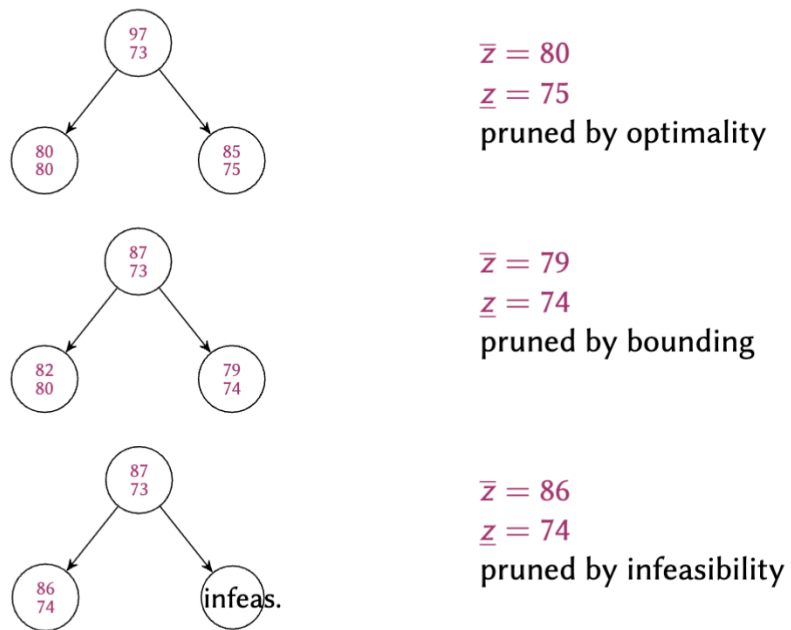


Figure 6: example on how